

Software Requirements Specification

1. Project Overview

Project Name

EN2H Booking Management API

Description

The Booking Management API is a RESTful backend application developed using NestJS and PostgreSQL. It allows authenticated users to manage services while allowing customers to create service bookings without authentication.

2. Objectives

- Implement JWT Authentication
 - Manage Services
 - Manage Bookings
 - Enforce Business Rules
 - Demonstrate NestJS best practices
-

3. Functional Requirements

Authentication

- Register
 - Login
 - JWT Authentication
-

Services

- Create Service
- Update Service
- Delete Service
- View Services

Only authenticated users can perform management operations.

Bookings

- Create Booking
 - View Bookings
 - View Booking by ID
 - Update Booking Status
 - Cancel Booking
-

4. Business Rules

- Booking must belong to an existing service.
 - Booking date cannot be in the past.
 - Duplicate bookings are prevented.
 - Cancelled bookings cannot be marked as completed.
 - Customers can create bookings without authentication.
-

5. Non-Functional Requirements

- Secure Authentication using JWT
 - Input Validation
 - RESTful API
 - PostgreSQL Database
 - Prisma ORM
 - Docker Support
-

6. Technologies

- NestJS
 - TypeScript
 - PostgreSQL
 - Prisma
 - Swagger
 - Docker
-

7. Database Design

Include your three tables.

User

Service

Booking

```
schema.prisma X
prisma > schema.prisma
1  // This is your Prisma schema file
2
3  generator client {
4    provider = "prisma-client-js"
5  }
6
7  datasource db {
8    provider = "postgresql"
9    url = env("DATABASE_URL")
10 }
11
12 enum BookingStatus {
13   PENDING
14   CONFIRMED
15   CANCELLED
16   COMPLETED
17 }
18
19 model User {
20   id      Int      @id @default(autoincrement())
21   email   String   @unique
22   password String
23   createdAt DateTime @default(now())
24   updatedAt DateTime @updatedAt
25 }
26
```

```

26
27 model Service {
28   id          Int          @id @default(autoincrement())
29   title       String
30   description  String
31   duration    Int
32   price       Decimal      @db.Decimal(10, 2)
33   isActive    Boolean      @default(true)
34   createdAt   DateTime     @default(now())
35   updatedAt   DateTime     @updatedAt
36
37   bookings    Booking[]
38 }
39
40 model Booking {
41   id          Int          @id @default(autoincrement())
42   customerName String
43   customerEmail String
44   customerPhone String
45   bookingDate  DateTime
46   bookingTime  String
47   status       BookingStatus @default(PENDING)
48   notes        String?
49
50   serviceId    Int
51   service      Service      @relation(fields: [serviceId], references: [id], onDelete: Restrict)
52
53   createdAt    DateTime     @default(now())
54   updatedAt    DateTime     @updatedAt
55
56   @@index([serviceId])
57   @@unique([serviceId, bookingDate, bookingTime])
58 }
59
60

```

8. Future Improvements

- Refresh Tokens
- Unit Testing
- Role-based Authorization
- Email Notifications